CRR Binomial Pricing Model
2-Layer Architecture Analysis

How the Deep Scholar CRR Model Answers CS495 Project 6

CS495 Capstone | Deep Scholar Project | Bellevue College | Spring 2026

Project 6 Central Question:

How do you build a trading / quoting strategy that maximizes expected profit under realistic market microstructure and biased crowds? When does the majority become systematically wrong, and how can a model exploit that while controlling risk?

Answer: The CRR no-arbitrage engine (Layer 1) establishes the theoretical fair price. Layer 2 detects when the crowd deviates from it and sizes positions accordingly.

The Core Mapping

The Cox-Ross-Rubinstein (1979) paper proves that any option price deviating from the no-arbitrage binomial lattice price creates an exploitable edge. The CRR model computes V_model from fundamental parameters (S, K, r, sigma, T, N); the market quotes V_market via the bid-ask mid. The mispricing edge = (V_model - V_market) / V_market is the same p - q gap Project 6 asks you to exploit.

Options contracts ARE prediction market contracts. 'Will AAPL close above $150 at expiration?' is structurally identical to a Kalshi binary event contract. The CRR risk-neutral probability p* = (e^(r*dt) - d) / (u - d) is the theoretically correct implied probability under no-arbitrage. When the market-implied IV (encoding q_market) diverges from realized volatility (the physical-measure analog), the crowd is systematically biased -- exactly the regime Project 6 asks you to detect and exploit.

Project 6 Requirement 1: True p Estimator

Logistic regression / gradient boosting on engineered features; ensemble probability calibration (Platt scaling / isotonic).

Deep Scholar Implementation:

p_estimator.py trains XGBoost on 5 years of AAPL historical ITM/OTM outcomes with Platt scaling calibration via CalibratedClassifierCV (cv=3), producing p_independent -- an empirically grounded true-p estimate. Features include moneyness, log_dte, IV, RV30, rv_iv_spread, half_spread, and log_volume. Brier scores achieved: CALL 0.1077, PUT 0.1028 vs naive baseline 0.25. Skill scores: CALL 0.569, PUT 0.589 -- confirming genuine predictive power beyond the market-implied probability.

Project 6 Requirement 2: Microstructure-Aware Trading Policy

Enter / exit based on edge p-q; size based on Kelly or constrained utility (max drawdown, VaR); avoid trading when spreads/fees erase edge.

Deep Scholar Implementation:

micro_cost.py models real transaction costs from AMD Fidelity ticket data: round-trip cost = 2 x half-spread + 2 x $0.0065/share (brokerage fee, confirmed from $6.50/10-contract tickets) + 0.1% x mid (slippage). compute_net_edge() deducts this from raw edge before any trade decision. policy.py gates on net_edge > min_edge with a 5% VaR limit per trade and a 15% cumulative drawdown halt. Kelly fraction f* = (p*b - q) / b with quarter-Kelly used in backtest for conservatism. The CRR paper assumes frictionless markets; Layer 2 adds friction back to answer when theoretical edge survives real costs.

Project 6 Requirement 3: Crowd Bias / Regime Detector

Design indicators like price momentum + volume spikes (herding proxy), mispricing persistence, longshot overpricing proxies. Condition policy: normal regime = trade small edges; herding regime = high-conviction only.

Deep Scholar Implementation:

bias_detector.py operationalizes the behavioral hypothesis using the RV-IV spread as the key signal. When implied volatility (encoding q_market) exceeds realized volatility by more than 10%, the crowd is herding -- overpricing risk in the same way the favorite-longshot bias overprices tail outcomes in prediction markets (Snowberg & Wolfers 2010). Over 2016-2020 AAPL data: 104 of 1,223 days (8.5%) flagged as herding, concentrated around COVID volatility spikes (Dec 2020: Dec 18-23 all herding). Regime-conditional minimum edge: normal = 2%, herding = 8% -- directly implementing Project 6's instruction to raise the bar when the crowd is biased.

Project 6 Requirement 4: Backtesting and Simulation

Deliver a reproducible backtest: P&L distribution, Sharpe-like metrics, max drawdown, hit rate and calibration (Brier score).

Deep Scholar Implementation:

backtest.py implements walk-forward validation: 12-month training windows, 3-month test windows, rolling forward across the full 2016-2020 dataset. Quarter-Kelly position sizing provides the responsible risk controls the spec requires. A 15% cumulative drawdown circuit breaker halts trading. All four required metrics are reported per window and in aggregate. Outputs: pnl_curve.png (cumulative P&L + distribution) and backtest_trades.csv.

Backtest Results vs Project 6 Requirements

Metric	Put Result	Project 6 Asks For
Sharpe	1.094	Sharpe-like metric
Max Drawdown	$246,474	Max drawdown
Hit Rate	4.1%	Hit rate
Avg Brier	0.16	Calibration (Brier score)
Call Sharpe	-0.947	Sharpe-like metric
N Trades (P)	57,053	Reproducible backtest

The original CRR (1979) paper establishes three foundations your implementation relies on directly:

1. No-Arbitrage Pricing

V_model is the unique price that eliminates riskless profit opportunities. When V_market deviates from it, edge exists by definition -- not by opinion or sentiment. Layer 1's edge.py computes this directly for each AMD contract: edge = (V_model - V_market) / V_market. This is the rigorous theoretical justification for why p - q represents genuine alpha rather than noise.

2. Risk-Neutral vs Physical Probability

CRR's p* = (e^(r*dt) - d) / (u - d) is NOT the physical probability the stock goes up -- it is the probability that makes the discounted price a martingale under the risk-neutral measure. The market's implied volatility encodes this q_market. Your p_independent from p_estimator.py estimates the PHYSICAL ITM probability from historical outcomes. The gap between the two -- the behavioral wedge -- is exactly what Project 6 calls the crowd bias and asks you to exploit. The RV-IV spread in bias_detector.py measures this wedge directly: high IV relative to realized vol means the risk-neutral measure has drifted from the physical measure.

3. American Early Exercise as Decision Theory

CRR's backward induction rule -- V = max(intrinsic, continuation) -- is the same decision-theoretic structure as Kelly sizing. At every node, the agent compares the value of acting now vs waiting. Kelly's f* = (p*b - q) / b makes the same comparison: trade now if edge exceeds cost, otherwise wait. The backward induction tree in Layer 1 and the Kelly gate in Layer 2 are the same optimal stopping problem expressed at different levels of abstraction.

Connection to Behavioral Economics Literature

Project 6 cites Snowberg & Wolfers (2010) on the favorite-longshot bias: bettors systematically overprice low-probability outcomes. In options markets, the analog is volatility risk premium -- implied vol (q_market) persistently exceeds realized vol (physical p), meaning option sellers are systematically overpaid. The AAPL data confirms this: 91.5% of days in normal regime with IV moderately above RV, and 8.5% in herding regime where the crowd overprices risk dramatically. Your bias_detector.py is empirically measuring the same phenomenon Snowberg & Wolfers identified in horse racing, applied to equity options.

The crowd bias function from Project 6 -- q = f(p, sentiment, liquidity, attention) -- maps directly to the RV-IV spread model: IV encodes sentiment and attention (the crowd's implied vol), RV encodes the physical signal (realized returns). When sentiment (IV) drifts far from signal (RV), the crowd is in a herding regime and the minimum edge threshold rises to 8% to filter out noise trades.

Project 6 Deliverable	Status
Research report (10-20 pages)	**Not yet written**
Code repository	**Complete -- pushed to GitHub**
Calibration curves (dashboard)	**Generated: calibration_curve.png**
P&L curves (dashboard)	**Generated: pnl_curve.png**
Regime analysis plots	**Generated: regime_analysis.png**
Final presentation	**Not yet built**
Ethics / responsible gambling	**Planned in PLAN2.md -- not written**

Architecture Summary

The two-layer architecture maps cleanly to Project 6's model framing:

Project 6 Component	Layer	Module	Output
True p estimator	Layer 2	p_estimator.py	p_independent, call/put_model.pkl
Market q proxy	Layer 1	edge.py	V_model, mispricing edge
Bias / regime detect	Layer 2	bias_detector.py	regime_analysis.png
Microstructure costs	Layer 2	micro_cost.py	net_edge per trade
Calibration	Layer 2	calibration.py	calibration_curve.png
Trade decision policy	Layer 2	policy.py	TRADE / NO TRADE signal
Walk-forward backtest	Layer 2	backtest.py	pnl_curve.png, trades.csv
Probability calibration	Layer 1	simulation.py	kelly.csv, simulation.csv

Conclusion

The Deep Scholar project fully implements the four technical components Project 6 requires: a calibrated p estimator (XGBoost + Platt scaling, Brier 0.103-0.108), a microstructure-aware cost model (confirmed from real AMD Fidelity tickets), a regime-conditional crowd bias detector (8.5% herding days identified over 2016-2020), and a walk-forward backtest with all required metrics. The CRR binomial model provides the no-arbitrage theoretical foundation that distinguishes this from a pure ML approach -- it grounds the edge signal in option pricing theory rather than empirical curve-fitting alone.

Remaining deliverables are the written research report (10-20 pages), the ethics and responsible trading section, and the final presentation. The code repository and experiment dashboard are complete and reproducible via 'make run-all'.